**Bachelor thesis**

for

**Mr. Vincenzo Ceccarelli Grimaldi**

# Development of a Cross-domain Knowledge Base for Cyber Intelligence in Smart Grids

Entwicklung einer domänenübergreifenden Wissensdatenbank für Cyber Intelligence in Smart Grids

Supervisor: Ömer Sen, M.Sc.
Dennis Van der Velde, M.Sc.

Datum der Antrag: 08.01.2021

Datum der Abgabe: 17.01.2021

(Univ.-Prof. Dr.-Ing. Andreas Ulbig) (Chair of the commission on examination affairs)

**Herzlichen Dank An Herr Klaus Dreher, der Vorort war in IAEW und der mich gerüstet hat, mit alles was man benötigt, damit ein Reibungslos Abgabe Prozess sich Stattfindet. Am letzten Tag, Heute 17.01.21, Vielen Dank nochmal. Vincenzo Ceccarelli Grimaldi**

**Contents**

**Table of Contents**

**Abbreviations**


**1. Instruction** 10

**1.1 Motivation** 10

**1.2 Structure Objective** 11

**2 Theoretical Background** 14

**2.1 Network Protocols and Energy Grids** 14

**2.1.1 Network Protocols and Systems** 14

**2.1.2 Smart Grids** 16

**2.1.2.1 Grids and ICT** 16

**2.1.2.2 The Cloud Infrastructure** 17

**2.1.3 Smart Grids - Transmission vs Distribution** 19

**2.1.4 The transmission Grid** 20

**2.1.5 The Distribution Grid** 20

**2.1 Information Communication Technologies ICT** 21

**2.2.1 ICT** 21

**3. Models and Graphs** 23

**3.1 Graphs In General** 23

**3.1.1 Database: the NoSQL Approach** 24

**3.1.2 Why NoSQL** 25

**3.1.3 Neo4j Access Patterns** 27

**3.1.4 Traverser API** 28

**3.2 A Brief On-the-market Entities relationship models that in spirit convey the Cybersecurity approach** 29

**3.3 Motivation for a Graph Database** 30

**3.3.1 Neo4j Caching Layers Use Cases** 30

**3.3.1.1 Advanced Persistent Threats (APTs)** 31

**3.3.2 Cypher** 31

**3.3.3 State of Triggered Event** 33

**3.3.3.1 Cyber Vulnerabilities and Attacks** 35

**3.4 Software Solutions and Components** 36

**3.4.1 Information Systems topologies - Smart Grid** 36

**3.4.2 Server Side** **36**
**3.4.3 Conditional Topological Search Analysis** **37**
**4. Approach, Results and Discussion** **38**
**4.1 Understanding the Blackbox sequential steps** **38**
**4.1.1 Link analysis explores relationships in cybersecurity** **38**
**4.1.2 Positives and Negatives** **38**
**4.2 Understanding the Dataflow within the Blackbox** **41**
**4.3 Similarity Algorithms** **41**
**4.3.1 Understanding the Dataflow within the Blackbox** **42**
**4.3.2 Interpretation of the Dataflow** **42**
**4.4 Concluding results** **46**
**5 Conclusion** **47**
**5.1 Summary** **47**
**5.2 Future work** **48**
**Bibliography** **50**

## List of Abbreviations

| | |
|---|---|
| **SSL** | Secure Sockets Layer |
| **SSH** | Secure Shell |
| **UML** | Unified Modelling Language |
| **PHP** | PHP Hypertext Preprocessor |
| **API** | Application Programming Interface |
| **BFS** | Breadth-first Search |
| **HMI** | Human-Machine-Interface |
| **AC** | Alternating Current |
| **HVDC** | High-Voltage Direct Current |
| **HV** | High voltage |
| **MV** | Medium voltage |
| **LV** | Low voltage |
| **RAM** | The Random Access Memory |
| **CPU** | Central Processing Unit |
| **CSV** | Comma-separated Values |
| **IoT** | Internet of Things |
| **DoS** | Denial of Service |
| **DDoS** | Distributed Denial of Service |
| **TDoS** | Telephone Hot-line Denial of Service |
| **RCE** | Remote code execution |
| **MitM** | Man in the Middle |
| **PLC** | Programmable Logic Controller |
| **VPN** | Virtual Private Network |
| **TTL** | time-to-live |
| **OSI** | Open Systems Interconnection |
| **IED** | Intelligent Electric Device |
| **DER** | Distributed Energy Resources |
| **ICMP** | Internet Control Message Protocol |
| **HTTPS** | Hypertext Transfer Protocol Secure |
| **HTTP** | Hypertext Transfer Protocol |
| **NAT** | Network Address Translation |
| **DNS** | Domain Name System |
| **MTU** | Master Terminal Unit |

| | |
|---|---|
| **RTU** | Remote Terminal Unit |
| **SCADA** | Supervisory Control and Data Acquisition |
| **ICT** | Information and Communication Technologies |
| **SaaS** | Software as a Service |
| **PaaS** | Platform as a Service |
| **IaaS** | Infrastructure as a Service |
| **BYOL** | bring your own license |

Statutory Declaration in Lieu of an Oath

Vincenzo Ceccarelli Grimaldi 353970

Name, First Name Matriculation Number

I hereby declare in lieu of an oath that I have completed the present bachelor thesis entitled

**Development and Verification of a Rule-based Intrusion Detection System Against Coordinated Cyber Attack Scenarios in Distribution Grids** independently and without illegitimate assistance from third parties (such as academic ghost-writers). I have used no other than the specified sources and aids. In case that the thesis is additionally submitted in an electronic format, I declare that the written and electronic versions are fully identical. The thesis has not been submitted to any examination body in this, or similar, form.

City, Date Signature

**Official Notification:**

**Para. 156 StGB (German Criminal Code): False Statutory Declarations**

Whoever before a public authority competent to administer statutory declarations falsely makes such a declaration or falsely testifies while referring to such a declaration shall be liable to imprisonment not exceeding three years or a fine.

**Para. 161 StGB (German Criminal Code): False Statutory Declarations Due to Negligence**

(1) If a person commits one of the offences listed in sections 154 through156 negligently the penalty shall be imprisonment not exceeding one year or a fine.

(2) The offender shall be exempt from liability if he or she corrects their false testimony in time. The provisions of section 158 (2) and (3) shall apply accordingly.

I have read and understood the above official notification:

City, Date

# 1 Introduction

## 1.1 Motivation

Electricity is essential to our contemporary economy and lifestyle. Due to this fact, most of our societies depend on electric structures that were built decades ago. Most of these ineffective systems, compared to today's standards, lack the technology and cannot offer a suitable response to today's urgent need of maintainability and scalability issues that arise at any given year. There is an estimated $13 billion investment that is required in Electric infrastructure over the following 20+ years [1]. This imposes the actual need and makes room for opportunities to shift towards a carbon efficient energy system, where our widgets and gadgets are better synchronized and scheduled for peak demand and for zero consumption time periods. The buzz word "Smart grids" will help on this transition, from the old state to a new current, more dynamic and self-healing tactics, that by enlarge will facilitate the scalability component that it is in need for our rapid demand of energy. This is where our Grids take a smart approach. A smart grid is a digitized energy network, with state-of-the-art software, bringing electricity in an optimal way from source to consumption. But this is achieved by integrating collected data, wired and signal-driven telecommunications and power technologies with the existing electricity systems integrated all into one source making it easy to overlay a control schema and regulate all inputs and outputs at any given moment. There are numerous benefits of what a smart grid consists, which mainly focuses on the constant self-efficiency and reliability of the electric supply itself.

Integration of our existing grid layout all the way to the adaptation and incorporation of the "renewables" into existing networks is the current task at hand. Supporting the development of electric vehicles and devices, from trains to ships at scale is one of the background themes "Smart grids" have to take responsibility for. These are interesting new solutions and optimizations for customers to implement their electricity consumption and above it all their reduction of carbon emissions. Cisco, one of the industry leaders in Hardware and Software Development as well as information technology, networking, and cybersecurity, does estimates Internet of Things (IoT) devices will take up more than half of connected devices roughly by 2022 (around 14 billion artifacts) which will serve as a catalyst for improved Distributed electric energy and its network security [14]. This is one of the driving factors to increase the study in the corresponding field in the IoT Security World, which itself

has a vast terrain of unexplored simulations and optimizations that engineers will have to inevitably conduct to put a safeguard in out collective delivery pipeline from electric energy to corresponding data that is transferred back and forth within and between networks.

Smart grids are not only about developing and improving existing infrastructure, it also embodies the realization of the potential solutions in the most effective and efficient manner. The World Economic Forum September 2010 reports [2] that fast-tracking prosperous smart grids modeled developed in partnership with Accenture and industry experts [2] from across the smart grid value chain, outlines conditions for success and possible challenges that might arise in this transition. Administrations and institutions, from governments to local communities, public and private alike, are increasingly identifying the actual worth of "Smart Grids". As an example, China aims at building a strong smart grid by 2020 [3]. There is a profound push to adhere to current wave of the Green world as it is currently underway in the developed world, including Germany. To ensure that public and private money is spent effectively, "Smart grids" network models and simulations must be successful and have a positive outcome in testing and evaluating all aspects.

## 1.2 Structured Objective

As mentioned, Smart grids are essential if we're going to achieve and adhere to some of our climate change objectives. In retro perspective, smart grids are a cohesive and enticing coupling of the whole electrical system itself, into one interconnected, supply-demand-control-energy system. Deploying and implementing Smart Grids at scale will be challenging and is the hardest problem we are going to face, but successful simulations of Smart Grids Network Models can set the direction. At our current Era, the data that the Power lines produces, is vastly unstructured and is growing at an unforeseen pace and most of its storage systems and facilities have a Relational Structure. Relational Databases happen to be a not-optimal design for unorganized and unstructured flow of data due to Redundancy and Performance degradation problems. NoSQL [11] is one approach to heal some of the entity relational model problems as we discover the open source graph database Neo4j [11]. Also, Neo4j will lend itself for a simulation application in power grid networks and IT environments to understand the notion of relationships and how to integrate the flow of data. Talking about the Smart Grid from the Interconnectivity perspective, our Information Technologies from the hardware to the

software, will not only sustain the Smart Grid to be automatically observable and controllable but moreover lay the topology in such a manner that is maintainable at scale. People at the forefront of technology will play a critical role in creating innovation, reward schemes that align incentives across the electricity value chain and encourage utilities and their partners to develop new technologies, operating and business models while acknowledging the risks involved enfacing cybersecurity. Smart Grid network Models will eventually need to develop greater consumer insight and systematically engage with End-Consumers at all times. Utilities will need to develop exciting new business models, attract everyday customer value propositions and ways to communicate them clearly to ensure their adoption. With regards to the collected data that is to be processed and modelled, is generated by an IoT device, which is intended to be transferred and controlled from-and-to their requested destination. The malicious activity will be assumed, used and simulated to mimic compromised devices that may resemble some of the most common attack vectors seen on the Internet and Intranet. Considering and applying certain guidelines and paradigms used in other industries outside of cyber security, the aim is to construct a blueprint with interchangeable components built on the abstraction to passively collect and analyze IoT network traffic data gathered by a network gateway device which is ultimately stored in a specific database for further analysis. This data is subject to be used, as we strive to identify compromised and potential anomalies, that might show up in various different environments. Signaling different patterns and schemes, that might be identified as breaches or intrusions within the network itself are detected as it is collected and stored in a database.

As we gear towards a sustainable environment, electricity plays a more and more important part in society. Behind all this, the incentive is to provide effective and efficient electric supply, including voltage transformation, electric power transmission and ultimately optimize its delivery pipeline. Progressively, researches lead the study on smart grids, which is also known as the topology of the electric system. This paper aids to explain and investigate the study of power grid system from the perspective of database management and execution environments. The aim of the following discussion is related with the overall end-to-end usage consumption as well as key elements related to graph databases. This combination would enable an overall cohesive argument to recognize and produce a strong argument for a positive discussion around graph databases when compared with traditional databases used in the Smart Grids of today. This approach hence allows advances of a highly dynamic and adaptable models for data visualization and data analytics. From a structure point of view, the discussion is divided into two main sections: the initial part highlights an overall platform in need of Smart

grids and the second part takes an approach into the Storage facilities and its visualization. Smart Grids will drift into the state-of-the-art research and adhere to some of its principles to set the stage for its own good, as it dynamically wants to preserve a Scalability and Maintainability feature, which are crucial for todays challenges as will be discussed in the following thesis.

The ultimate objective of this thesis is to create a feasible and efficient solution to analyses sets of data and detect Advanced Persistent Threats (APT) [12] attacks in a reliable way. The graph database approach demonstrates a way to analyses multiple interrelated "events" in a straightforward process. The mental model is as follows: The introduction and motivation are presented in Chapter 1. Within Chapter 2, outlines the necessary background of Energy grids, how it becomes a smart grid and some of its protocols and networks that make up the other side of a grid, being its "digital part", also setting solid foundation on the Transmission vs Distribution side of the Energy Grids themselves and how ICT plays a role in its continuous development. In Chapter 3, the notion of graphs will be assessed and the motivation behind it, which will answer some questions to recurring problems that arise in Networking and its topology. Setting the stage for a NoSQL approach, ultimately leading to a Neo4j modeling which might suit best todays demand for interconnectivity and visualization. Chapter 4 presents the outcomes of its dynamic fusion, the topology of the Smart Grids and use cases for Cybersecurity and threats that we might face in deploying this architecture. In the final Chapter 5, a conclusion is induced from the Distribution lines and finally an outlook on potential progress and advancements that may not actually be so future distant.

# 2 Theoretical Background

## 2.1 Network Protocols and Energy Grids

### 2.1.1 Network Protocols and Systems

The actual functionality of interconnected networks and their used protocols are to be understood as a well-defined set of rules that govern how data is delivered between different devices in the same or between networks. Fundamentally, it lends itself to connected devices with each other, regardless of their internal processes, assembly or design. Network protocols make it feasible for any sort of device to interact and access data that neither local area networks (LAN) nor wide area networks (WAN) [12] would function the way they were built for, without the use of our conventionally used protocols

**An overview of the Protocols that are most profoundly used in the industry today [13]:**

- **Internet Control Message Protocol (ICMP)** is widely used to determine if data has reached its final destination using timestamps in a chronological fashion. Routers that work as Network devices use the ICMP protocol. ICMP is vital for reporting failures and continuous testing which can also be utilized to perform a distributed denial-of-service (DDoS) attacks.
- **Transmission Control Protocol (TCP)** is a telecommunications standard that permits request programs and computing physical devices to exchange packets that are known as messages over a particular network.
- **Internet Protocol (IP)** is a set of rules that consolidate how data-transfer should be delivered over the public and or privet network (Internet/intranet). It works in conjunction with the transmission control protocol (TCP), together they are referred to as TCP/IP.
- **Ethernet Cables** are used to deliver the data within an internet connection, also used to connect devices to a local or remote network. They plug into Ethernet ports on a variety of devices.
- **Addressee Resolution Protocol (ARP)** is a telecommunication protocol used for determining the link layer address, for example a MAC address, which is often times associated with an assumed (imbedded in the packet) internet layer address, typically in the Dual Stack which has the IPv4 address and IPv6 Address.

- I**nformation and Communication Technologies (ICT)** includes all devices and protocols including hardware and software that store, process, transmit, filter, convert, duplicate, or receive electronic data.
- **SCADA** stands for 'Supervisory Control and Data Acquisition' and defines the association of large spatially distributed ICT systems mainly in the Power industries in a hierarchical and synchronized Manner. A SCADA system embodies various entities such as Master Terminal Unit (MTU) with an associated Human-Machine-Interface (HMI) and multiple Remote Terminal Units (RTUs) with connected Intelligent Electric Devices (IEDs).
- **Master Terminal Units (MTUs)** have a sole purpose, to do monitoring and controlling and maintain overview of the whole system. It can be regarded as a central point of control. In larger plants using extensive infrastructure, there can be sub-MTUs and failover-MTUs.
- **Remote Terminal Units (RTUs)** consists of computer devices and interfaces that are between the real world and the SCADA system, these interfaces provide ongoing measurement values which also process signals from local sensors and control local actuators. An RTU is most of the time connected to one or multiple IEDs.
- **Intelligent Electric Devices (IEDs)** is a device that performs electrical protection functions, progressive local control and has the ability to monitor multiple processes running at the same time in parallel and above all, they can communicate directly to a given SCADA system.

### 2.1.2 Smart Grids

#### 2.1.2.1 Grids and ICT

In order to control and assess on a chronological fashion a smart grid, the Grid itself needs to be deployed with different kinds of modules and technologies, communicating with each other, connected and working collectively ensuring all tasks are performed without delay. This is where monitoring through sensors and IoT devices comes in place. This sets the stage to allow constant control thorough-out the whole energy distribution grid, its actual consumption that is subject to change across the year, and its modern storage as we produce non-stopping flow of data at an exponential rate. The integration of Powerline Grids and ICT actually lends itself to monitor is own flow of data and auto-efficiently optimize its very own operation systems within the Grid itself. As power consumption reaches its peak demand at unprecedented times due to volatile Distributed Energy requirements, Network topologies as for ICT is becoming an essential part Grids themselves. The usage of ICT helps to **regulate, manage and overall monitor-display the flow of information** in "Smart Grids". ICT is where network traffic is captured and assessed, where the consumption and delivery pipelines at certain time intervals could be seen to have a range from micro seconds even up to days, sending packets back and forth with updates from the Grid [5]. **Traditional Grid** permits and fuses different **Grid** levels such as European constructed in a four-level hierarchy assembly, which is divided between transport and distribution areas, as well as having different facilities in charge of higher and lower voltages, all the way to the target costumer. Figure 2.1.1 shows a scheme of the overall energy grid.

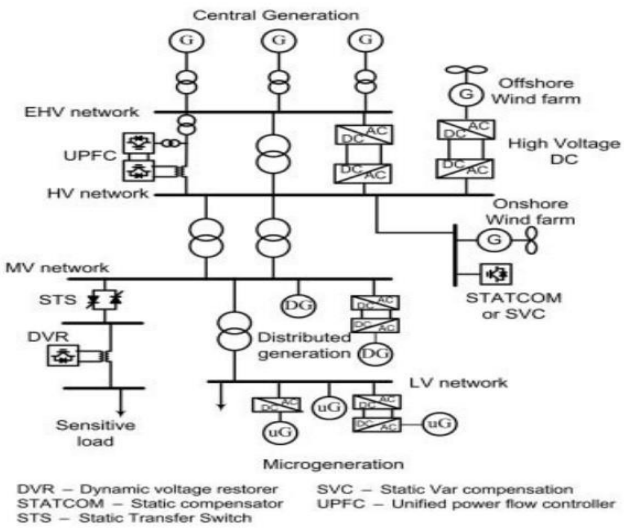


**Figure 2.1.1:** European Electric Grid scheme [6]

Smart Grids on the other hand are a construction of embedded systems and Optimized Protocols to make the actual Grids self-healing in response time which should function optimal for peak demand and for Zero consumption periods, controlling and adjusting load flow in various bus systems, delivering active and reactive power all the way from the Generation plants to the End-User.

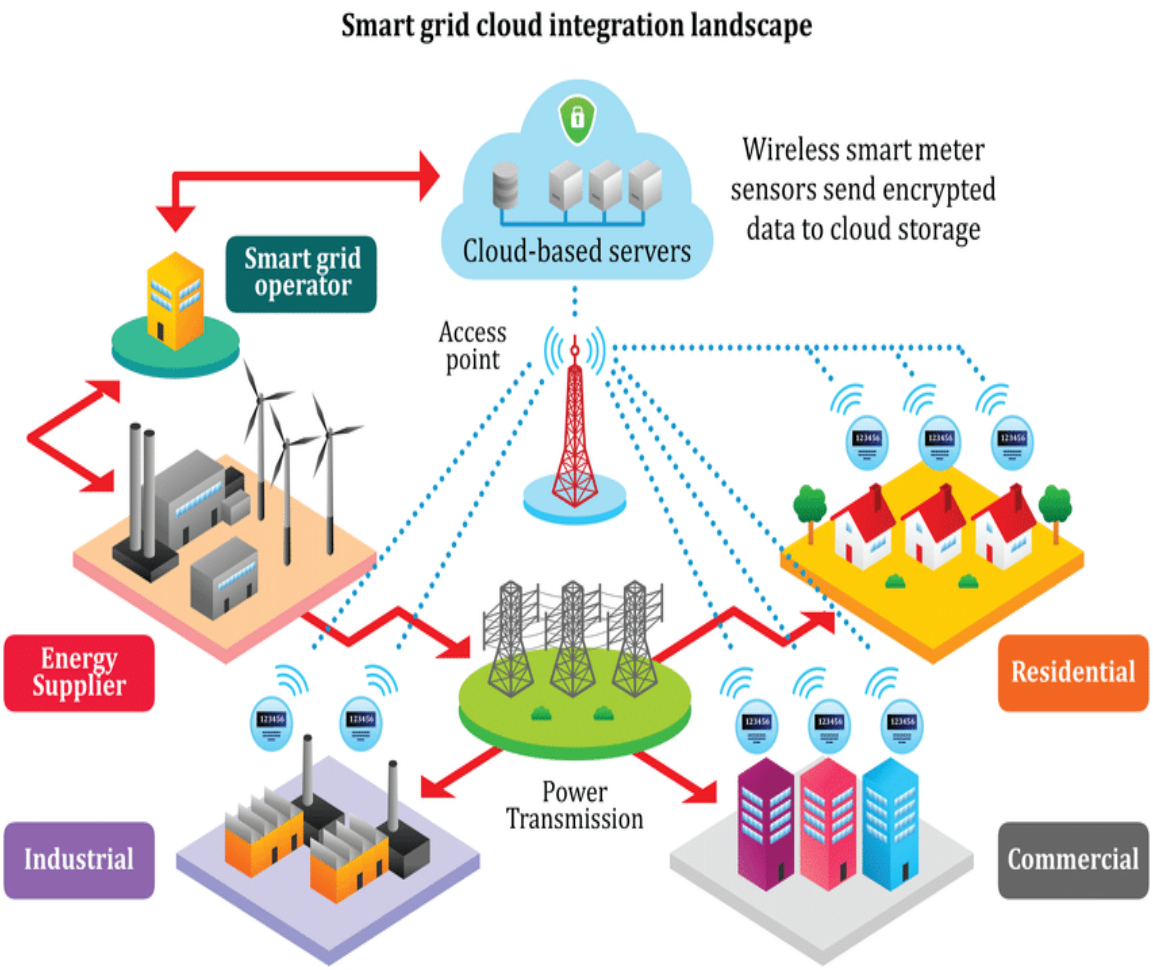


**Figure 2.1.2:** Simplified Smart Grid scheme [7]

### 2.1.2.2 The Cloud Infrastructure

To help us synthesize the workload, we must break it down into various degrees of complexities and isolating certain domains for better understanding of the Grid topology and make the most usage out of the current infrastructure.

- Cloud Computing Layer [9] offers managed services to its users (consumers) via web-based tools and applications. The layout of these managed services can be dismantled in 3 categories. These categories are: Software as a Service (SaaS), Platform as a Service (PaaS) and Infrastructure as a Service (IaaS).
    - A brief Summery
        - The model of a SaaS subscription provides software "like" services (Email, virtual desktop or apps) to its clients and using BYOL (bring your own license) licenses are deployed either on-Premis or in the hosting Cloud provider.
        - In PaaS applications delivers the feature of "runtime environments" in the actual physical storage device which is either rented or purchased from the Cloud provider.
        - IaaS provides an interface such as Virtual machines or containers that enable a pre-configured staged environment to its users (consumers) where data can be processed from PLC's sensors, smart meters readings, and coping with the dataflow at high rates, buzzword "Elastic Services or better known as Serverless Programming".
- Fog computing layer [10] act as a proxy layer between Cloud and end its users (consumers), to lessen the load and increase the overall performance of the hosting Cloud. Making this a powerful concept in terms of Power reliability that improves automated control, self-healing systems, ultimately resulting in fewer outages prognosticating better consumption periods. It also assists with the electric Power productivity which shortens the total energy usage, anticipates peak demand and therefore reduced losses, and the optimization of its maximum capacity utilization. And last but not least, bringing a Financial benefit that results in minimizing costs, improving delivery pipelines and timestamp-payloads that are used for synchronization.

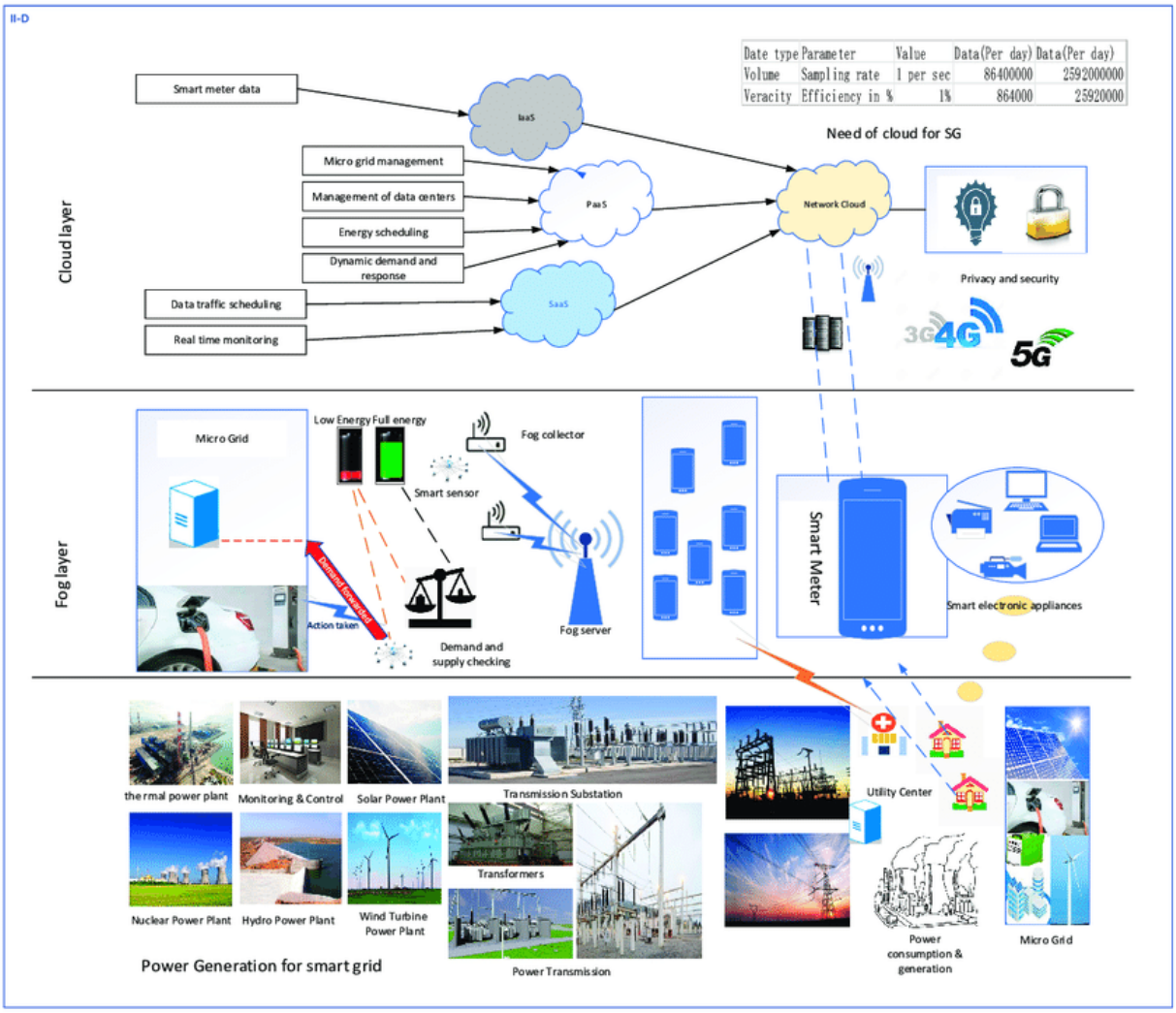


**Figure 2.1.2:** Fog Layer and the Cloud Layer of the Smart Grid scheme [8]

### 2.1.3 - Smart Grids - Transmission vs Distribution

There are multiple reasons why in today's energy industry sector, it is wise to forecast potential problems and run simulations within various Grid Topologies. Nowadays we are able to show and synchronize actual simulations on different distributed Grid scenarios aided by various IoT devices that help visualize the dataflow and enable our current power grids to integrate themselves better. Now, in one hand we have the transmission network. So far, we have discussed different scenarios of data transfer and solutions to be deployed at the distribution site. To further this analysis, our infrastructure itself spreads over a vast distance, using (WAN) [12]wide area network channels, is extremely subject to change not only because of weather conditions but also for Customer-time period demands. As one Example solar fields, need to change the direction of the PV panels, tilts depending on where the Sun is projecting its light. They also need to change the direction they're pointing at, depending on the time of

the year. Walking into the wind energy sector, you have onshore and offshore wind farms, that are key in this new ear of Eco-friendly environment. They change depending on the season, so we have to predict how much energy one is consuming or how much power you're generating at a certain day, depending on the previous days of the year. Grids themselves also have to Employ more energy storage units, depending on the time of the year, due to summer and winter seasons, the generation of solar power is also subject to change, one needs more energy storage for the excess power that is being generated in real time.

### 2.1.4 The transmission Grid

At the transmission network, it requires incredibly high level of control. Across Cities users have extremely high voltages running through these power lines and there is a need to control them at any given point in time. In general, there is a hunger for a complete control over the power lines and over the grid. The transmission network itself is spread over a very wide area, mostly rural areas, which means that you have power lines running for hundreds of kilometers without reaching a transformer or without reaching a repeater which increases the probabilities for failure. One may agree that “the distribution network” works hand in hand with the Transmission Network. It happens to be that because of privacy, we are not able to note well in advance people’s and different Industries total consumptions. This is why the distribution side is by definition very volatile and prone to failure if not dealt with anticipation.

### 2.1.5 The Distribution Grid

The Distribution network has always been a very high-risk environment, always in need of reliable monitoring systems and very accurate control protocols. However, these monitoring and control systems are complicated by definition if they are diploid at scale. In order to identify all the possible risks or disturbances in the transmission network, administrators need to obtain information from different sources in order to conclude possible failures within the system. RTUs and MTUs are to monitor the imports and exports coming in and going out of the “Smart Grid”. Then you have to monitor the failures as they occur over a period of time. This is where ICT comes in.

## 2.2 Information Communication Technologies ICT

### 2.2.1 ICT

As Mentioned, this is where information communication technologies come in, to help and make the delivery pipeline robust, so that as failures are anticipated, the correct people are notified and triggered events for failovers are put in place. Here, there is a need to monitor and measure the wind power, for example, in order to be able to predict the wind energy that we can generate at a certain day. Same goes for solar energy. There is a need to monitor the line to line power currents and their loads. There is a need to have remote infrastructure sending and measuring the transmission lines at the user side in order to anticipate the entire power demand. The Power Cable lines [13], for example, at remote locations, can automate monitors and the conductor temperature sensors so administrators can have Data being sent back and forth. As mentioned at times, the monitors need to tilt the towers in order to prevent Ice formation, for example, in order to avoid fire hazards or short circuit faults. And as we acknowledge the sense of remote infrastructure, considering thousands of sensors monitoring different phenomena, different parts of the grid, we are going to need to visualize this for better understanding.

One of the recurring questions in the Smart Grids are is “how frequently do we have to monitor the system”. Some of these phenomena can be measured yearly. Other phenomena can be sensed daily, or some of these phenomena must be actually sensed every one minute, every 30 seconds or even every microsecond. Another question is “what kind of technology offers the desired coverage”, as we start talking about the transmission network, one assumes distances of over more than 100 kilometers.

*Is it possible to provide this kind of coverage with wired cables*?

One must take into account the Cost for these wires, plus the maintenance overhead, plus the fact that you actually need to deploy them. Not forgetting the large distance, it might not be very practical to lay wires for hundreds of kilometers. One other possible option is to use wireless infrastructure, which is where the Cloud notion comes into play and the Graph interconnectivity does help resolve some of these issues.

In the United States alone, its roughly estimated that within the next couple of years, the average person will have more than ten electronic items per household, average of two and making it a challenge for our current Grid topology [14]. Of all those electric devices Cisco approximates that about half of them could be wired to the internet and therefore be identified

as IoT devices [14]. With the increasing number of devices at the average home and more importantly as the number of companies that produce these devices remains at a constant increasing pace, the quantity of data proportionally increases. One of the End Results of this massive flow of data, it correlates a potential that the devices are subject to be compromised by a malicious actor. In order to take security seriously, and for the betterment of the End user as a whole, the mechanisms that are used for security of these devices do not only have to include the software themselves but also a firewall like feature that once connecting to any network, especially the average home network, will not be subject of a scam or been spied on. By investigating a multiple framework utilizing data batches and aggregation of the composed data into a storage device, or a database designed to treat and model relationships between data as equally or more important to the data itself, with the definitive aim which is to identify malicious activity coming from any compromised IoT device (Neo4j, 2020d). By examining and visualizing the dataflow which is generated by any IoT device in a database, we will need to first understand the notion of how to integrate and maintain fine grained control of the data itself, which sole purpose is to ease the problem that the Smart Grid administration faces once new technologies constantly make themselves available in the market.

## 3. Models and Graphs

### 3.1 Graphs in General

Graphs are to be seen as general language for relating and analyzing entities with relations in interactions. This means that much rather than thinking of the state of the world or a given domain as a collected set of isolated data points, we are to think of it in terms of mapped networks and relations between these mapped entities. In other words, there is the underlying graph of relations among the entities and these mapped entities are related with each other, according to these mapped connections or the overall structure of the graph itself. There are many numerous kinds of data that can be certainly represented as graphs and modelling these graphical correlations should be taken into account. Relational structures of the underlying domain will allow us to shape much more correct, much more accurate models of the underlying phenomena or underlying data that is at hand. As for example, we can think of computer networks, disease pathways, infrastructure as well as "triggered events" can all be represented as graphs. Similarly, we can think of the classic social networks, economic networks, telecommunication networks, citations between different papers, internet as a giant network, in addition to ways how neurons in our brain are linked and ultimately how Grids evolve to become more sustainable and reliable, better known as Smart Grids is by definition a Graph Network.

Again, in the Smart Grid Domain we have inherently networks or graphs, and illustrating these networks will allow us to capture relationships between different objects or entities in these different domains. Last but not least, we can take data also known as packets that are being sent back and forth and represent them as relationships between different entities or nodes in a system. To further emphasize this notion, computer code software can also be represented as a graph, buzzword "calls between different functions" or as a "structure of the code captures by the abstract syntax tree". There are many different aspects of *natural graphs* or what we call "natural networks", where any underlying domain can naturally be represented as graphs. Social networks societies, a collection of seven billion individuals in connections between them communications between electronic devices, pager calls, financial transactions all naturally form graphs. And in order for us to be able to model these domains, we represent them as networks. Another subset of graphs that we are going to talk about is the *knowledge graphs*. This is a domain that can also have relational structure, and we can use graphs to represent that relational patterns within the modeled structure. So, for example, information

and knowledge is many times organized and linked. Software can be represented as a graph using timestamp payloads that are embedded every single time the functions are called. We can many times take data points and connect similar data points, a “Similarity Graph” being a subset of the Knowledge graph have natural relational structure like Smart Grids, mapping Usage consumption, Distribution Networks, as well as plotting Outages or Intrusion Detections can all well be represented as “knowledge Similarity graphs”.

Before we make the transition to the Graph Database, it is important to set a solid foundation, highlighting the reasons that will enable us to have a better comprehensive understanding of the technology *we currently have at the moment* and *what is in store for us that can be regarded a proxy for further development*.

### 3.1.1 Database: the NoSQL Approach

Most of the design patterns we are going to be discussing today actually apply to all NoSQL databases. Meaning that SQL happen to have as a single-strong-basic component: multiple-Tables with fixed rows and columns and on the other side, we have NoSQL that encapsulate Document Stores such as: JSON documents, Key-value: key-value pairs, Wide-column: tables with rows and dynamic columns, Graph: nodes and edges which will be ultimately used and serve as best candidate to solve some of the problems at hand. NoSQL is suitable for simple well organized big data instead of data with plentiful properties and labels. If we add relationships to NoSQL data, aggregate joining operation are needed at the application level, which is prohibitively expensive for large data volumes, overloading the CPU which is in charge of the joining and denormalization of the views to be rendered.

And as we walk back in time, a brief history of data processing to kind of set the tone, the mindset for the reasons why are we even looking at NoSQL. It's important that we understand this in the Grid System world is response-time driven. As we move forward, we are going to get into some of the common design patterns used in the industry now and throughout the last couple of years, and above it all, how graphs lend themselves to an optimal potential solution within one table structure, which eases some of the challenges for data visualization.

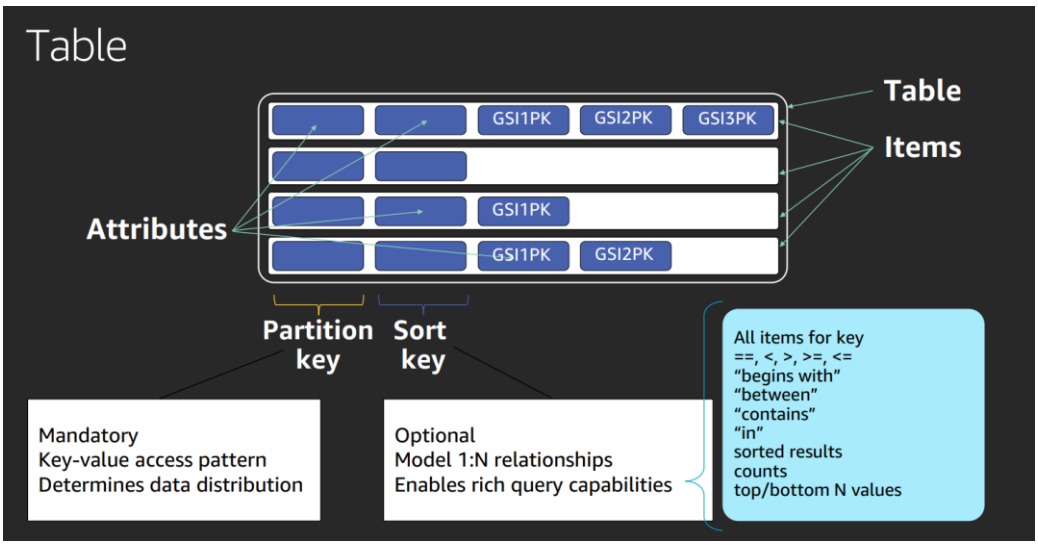


**Figure 3.1:** table NoSQL

### 3.1.2 Why NoSQL

There was a point-in-time in our current Era, where we used paper tape, magnetic tape as well as distributed block storage, random access file systems and along came 1970 or so the relational database. It's important to understand the actual reasons behind building and implementing the relational database, and one of the compelling reasons is that we did it because storage was expensive. Normalizing the data, reducing the footprint of that data on disk was very important. The relational structure that maps into a "data store" is a wonderful way to reduce the storage cost of your application though it increases the CPU costs since it has to perform complex queries which it must execute to produce the normalized views of data that your application consumes, also known as "Joining tables" and "group-by's". So now fast forward 30 - 40 years, current time spam, and the most expensive resource in the data center is actually the CPU. It's not the storage anymore. NoSQL are not made for complex queries, such as Inner joins and calculate values. It really breaks down to the OLTP application is an excellent application for NoSQL databases and good for us because it makes up 90 percent of the applications we build which end up representing very common business processes. If the administrators understand the access patterns very well, once they are repeatable, once they are consistent, then that's when we go to NoSQL. If not, then let's look at the SQL database, it's not obsolete but the Smart Grid Infrastructure would be best suited for a NoSQL approach. And it's just now we have a different choice against the SQL approach. The thought process behind OLTP and OLAP: Online transaction processing (OLTP) stores and processes data from transactions in near real-time. Online analytical processing (OLAP) utilizes expensive

complex range queries to analyze, often times chronological data from OLTP systems, in order to assemble views. [12]

| SQL | NoSQL |
|---|---|
| **Optimized for storage** | **Optimized for compute** |
| Normalized/relational | De-normalized/hierarchical |
| Ad hoc queries | Instantiated views |
| Scale vertically | Scale horizontally |
| Good for OLAP | Built for OLTP at scale |

**Figure 3.2:** Table SQL/NoSQL

This infrastructure, whether it's running on EC2 or on-Prem, it's irrelevant. It's is proportionate scalable and maps in manhours to the same cost, more being: Server updates, patching operating systems and rebuilding-software storage devices, failed nodes and many more. Electrical Engineers in the *Transmission Grid* should not be concerned with such. That's not core to the Energy Transmission business, but it is of the **Smart Grid**! Considering the *distribution Grid*. The fully managed aspect of NoSQL database services such as Neo4j is really the most powerful feature that you can have. In basic terms it is a Graph with nodes and edges. What we really mean is wide column key value store once you parse it as a JSON file, that supports a document attribute type.

One can restrict the access to the items on the table, to the table itself, to the attributes within the item. If the Smart Grids have an application process that's running against a data store that might have items that contain information Administrators don't want to have visible to order entry clerks or other personnel, then maybe they have annotated data, which is interesting, like maybe sales persons commission data and Security credentials such as for SCADA [13] or MTUs that administrators themselves want to have access only for sales managers, this is possible. Administrators can have those access patterns hit the table with different security permissions, which gives me a fine-grained access control over the data and what people can read. And then it's a back-plane service, which is really great when you talk about serverless programming.

One can have many items into one Node, and the items in the Node don't always have to have the same attributes, but they do have to have one attribute that uniquely identifies the item and that's the partition key. So, every attribute I insert into a Neo4j, the table must at least have a partition key attribute. It can very well also include an optional sort-key attribute. And when I do this, it gives me the ability to execute complex range queries against the items that are in those partitions.

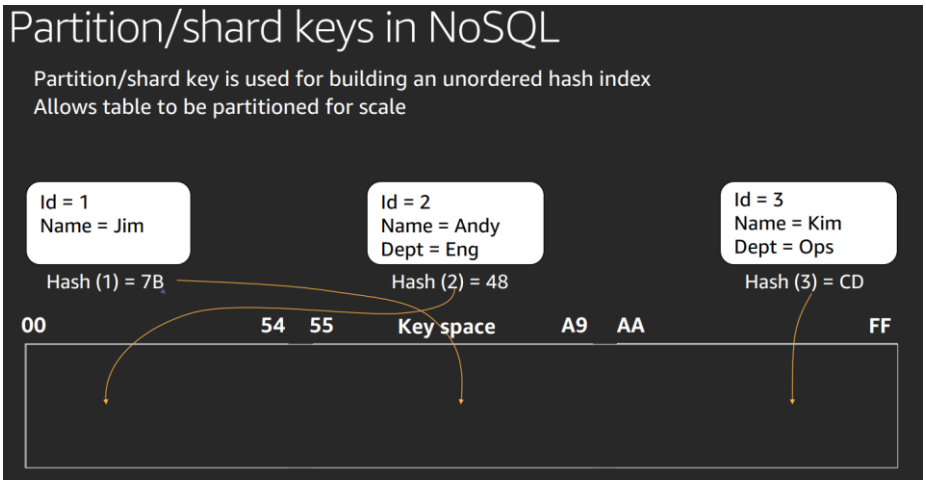


Figure 3.3 Unordered hash index Partition

Having a mental model of the partition as now a folder or a bucket that contains items and the sort-key orders the items within that specific folder. And so, when administrators query those partitions or those folders, I can execute those queries with complex range operators, making Neo4j optimal that maintains one-to-many relationships.

### 3.1.3 Neo4j Access Patterns

Neo4j [11], an open source graph database which is written in Java and Scala, gears towards generic graph models with common database characteristics such as ACID-transaction compliance. Typically, administrators want to model the table to support one of your primary access patterns with cluster support, runtime failover, and query language. One always wants that table to be able to query something that is interesting to the running application. It denotes the different roles by nodes with multiple labels, and embodies the relationships by paths between nodes. Partition keys are used to uniquely identify the item. They're also used to distribute the items across the key space. See Figure 3.3. Every Graph in Neo4j can be represented as one or more unique, logical key spaces, and we're going to distribute those items

by taking that partition key attribute and creating an unordered hash index (or a B Tree structure to some degree depending on the commercial host providing the service), laying these items out across this virtual key space. This is a way all NoSQL databases actually work, and as we scale the database, administrators are just going to chop that key space up and spread those items out across multiple physical storage-devices. So now when one query's the system, the administrators are going to always provide that partition key as that equality condition. In other words, the system knows exactly which storage node to get to go read the requested data. This is what makes all NoSQL databases fast and consistent at any scale. As an overview, there's automatic routing from the query to the exact storage node that needs to be served to service the request. Now, provided that sort-key condition, the query itself is going to go into the partition, selectively read the items that are co-located on the same storage node and sorted using that stored-key attribute. This is how NoSQL databases maintain that fast, consistent behavior. This is not unique to Graph Databases; all NoSQL Databases share this construct.

### 3.1.4 Traverser API

The Traverser API [11] allows administrators and users to traverse the graph at hand by means of callbacks. Users can hardcode an approach to go through a graph or subgraph for that matter, using detailed queries and algorithms, such as depth-first or breadth-first. There's a lot of differences between how one models data in NoSQL and how one models data in SQL databases. But the bottom-line is that all the data is relational. It's the same entity relationship model that we're going to build and that we're going to manage and ultimately that we're going to have to support with the application service. Regardless of what type of application that we're building, it embodies the same blueprint such as social networking, document management and Grid Structures. Every single application you can think of has some sort of data model that is relational in nature.

Walking into the domain of data encapsulation or data hiding, one may agree that in order to formulate a framework that allows for the capability to detect malicious or nefarious activity inside any network, it is wise to first look at principles and paradigms used in other industries. Through learning malicious activity closer to open source documents and adhering to some of the frameworks around this data will be an incentive and promote new ideas to further study cyberattack exposure. The following notion will allow for a better understanding by what method graphs and ultimately graph databases can be used to secure that networks and devices at the edge of the Internet, running on the Cloud or on-Prem. The proposed schema

will explore passive data collection decisions that allow for dataflow storage into a graph database to demonstrate the viability and resourcefulness that graph in general have in the IoT security space. By collecting and processing data passively from any sort of IoT device within a network and aggregate data from multiple devices into the target database, including normal and malicious traffic, is one way to simulate traffic and ultimately comprehend malicious activities. Two such tactics that may be explored are anomaly-detection by means of Statistical Examination in addition to entity link analysis, a concept of fraud-detection utilized to analyze relationships and "similarities".

Machine to machine, once computers have been hypervised using for example VMwareFusion or virtualBox, data broadcasted is often times conducted via http or https. Telemetry material details the device's interaction with publicly or privately available servers, including Headers that contain the destination port, destination DNS (Domain Name System) information and IP used for that request, also containing the amount of data in and out calculated in bits per second, also known as foundational Internet protocols, and other connection related data. Machine to machine communication is not the only information gathered but also payloads that may also contain the devices "traces and connections" all the way to the end user. This communication schemas include IoT protocols such as SSDP - Simple Service Discovery Protocols and mDNS (multicast DNS) [12] which also uses mac addresses that are used to recognize the devices in a network.

## 3.2 **A Brief On-the-market Entities relationship models that in spirit convey the Cybersecurity approach**.

- **CyGraph** – Mitre Enterprise Cloud Detection - The Mitre Corporation, that is federally funded, non-profit development platform, did construct a preliminary graph model tool called Cauldron (Mitre, 2019) used for Cybersecurity. Cauldron, which later transformed itself to a commercial product CyGraph, is composed by isolated data and events into an ongoing persistent document for decision making and situational awareness intended for intrusion detection, anti-virus warning, and other events like logins [14].

### 3.3 Motivation for a Graph Database

Multiple Tables have a complex set of queries that need to be executed right to *get a list of all my RTU, PLC, HMI, and their attributes*, making three different queries joining up to multiple tables, at scale, this will become problematic. In other words, this is why relational databases cannot scale because the CPU is being utilized the wrong way, hopping all over the disk, pulling data off of all these tables, sticking it together in a normalized view, and serving it up to the application layer. And on the flip side, when one needs to update that data, the actual data still lives in multiple places. Not walking into disarray from the Thesis but ACID transactions are to be considered and further studied. The reason for that is because of the data model that we're using with relational databases may have a better approach for us to avoid. On top of the visualization capabilities, Caching is one of the most important components in Neo4j because it directly affects read and write performance. Administrators will inevitably want to make hierarchical data structures. What is also wanted is to collapse data and build documents or collections of items in relation to one another like a graph within single partitions that represent these data hierarchies.

#### 3.3.1 Neo4j Caching Layers Use Cases

Neo4j provides two different types of caching layers that are commonly used. One is a file-buffer-cache that utilizes off-heap memory in order to be able to cache data stored on disk. The other caching property is an object cache. And now, instead of having to execute three queries with various degrees of complexity (*get a list of all my RTU, PLC, HMI, and their attributes*), I'm executing one simple query that says Select MATCH WHERE [11]. These are much simpler queries, much simpler access patterns, and you can see immediately why does the system scale better. When I use this type of hierarchical data model than this relational data model, because in the end, we are not executing complex operation to assemble the views, we are now just going and getting documents or collections of items out of single partitions. All writes and reads are done and performed through these caches, as mentioned, on order to improve throughput. All writes are written to the caches and data in these caches are put subsequently to durable storage within a particular partition. Instead of joining, it might be better to have it re-do for the application layer to be just read directly from the partitioned Mode Schemas and this is where Graphs come to the rescue, offloading the CPU, so that query-executions are done much faster skipping the joining phase. Neo4j also improves write performance by batching small writes.

#### 3.3.1.1 Advanced Persistent Threats (APTs) [14]

These attacks are a main topic in cyber security of computer information networks. I will strive to demonstrate a feasible and fast way to analyze real-world log-data in order to be able to detect breaches or breach attempts. By adapting well-known kill chain mechanisms and a combine of a time-series graph database functionalities, it is possible to create flexible attack profiles. Using this approach, it can be demonstrated that the graph analysis could successfully detect attacks by analyzing the simulated log-data of a simulated computer network. Considering the origin source of the log-data, the projected work should be capable to deliver sufficient performance for analyzing real-world data in near real-time. By using the computing power of the graph database, it is possible to classify the attacker and furthermore it is feasible to detect other affected system components. Running simulations, we believe to significantly reduce the detection time of breaches with this approach, and foremost react fast to new attack vectors. Attack vectors happen to be most intensely used on file-audit, windows events (logon, logoff) and firewall logs.

### 3.3.2 Cypher

Cypher, which is tightly coupled to the Neo4j Open source application, is a declarative database query language [21], similar in spirit to SQL query language. It is simple and expressive, resembling the natural way to represent patterns in graphs through ASCII art diagrams, using arrows and circled boxes. In this paper, we will mostly use the Neo4j Traverser-API [11]. The Cypher query language is often times used for the Shortest Path function. Before you can proceed with the traversal API, one needs to find the starting node. The Neo4j Core Java API has a single lookup technique that can be utilized to load nodes from the database itself—GraphDatabaseService.getNodeById(Long id). This method loads and puts in cache the node by its internal identifier, which is characterized as a java.util.Long number in Neo4j. One can assume that the ID which corresponds to 1 user node is known and stored in the static variables, for example Bus_Num_ID. See Figure 10.

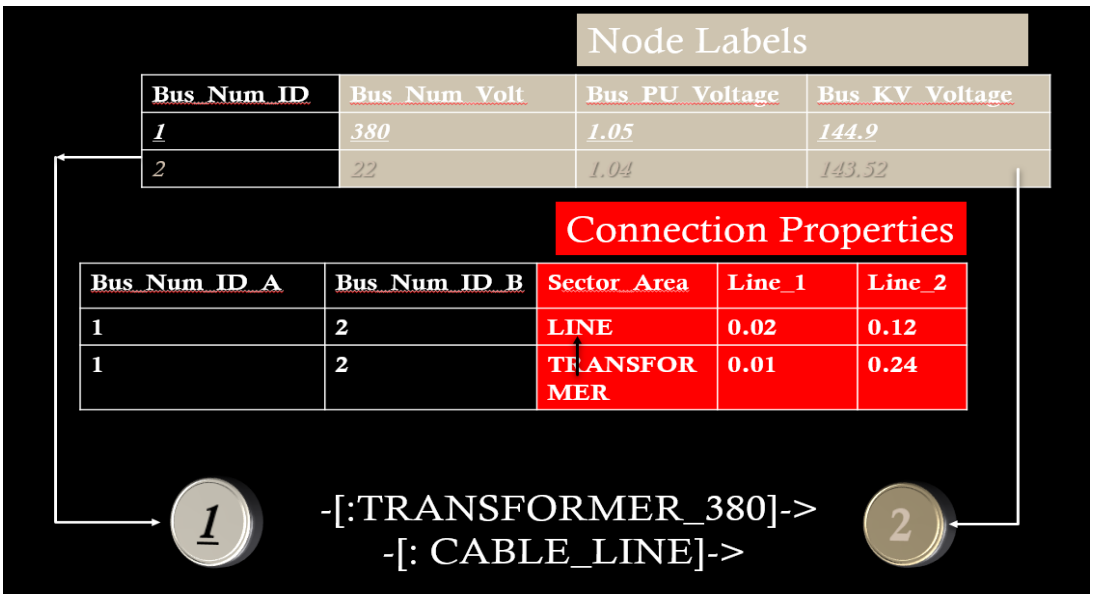


Figure. 10. Topology of Graphs Relationships

Figure 10 denotes a table with two nodes, and the second table on the bottom is the assembly table with connection information among node 1 and node 2. As it shows in the figure, two nodes can have multiple associated relationships with entirely different properties. The initial State Power Grid Network System, as one example, can be mapped as the initial state analysis which can be process for determining what equipment has power "if" power is provided to certain nodes. See figure 11. Each node denotes a physical bus with its corresponding ID. The output is a list (or tuple) of all the nodes that are interconnected to these starting nodes, through one or more sequential paths.

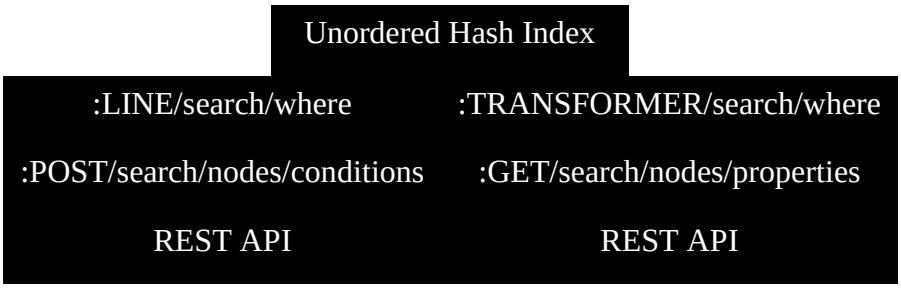


Figure 14 REST APIs provide GET and POST function for conditional search. These functions can be called either from CLI PowerShell or directly from Neo4j client.

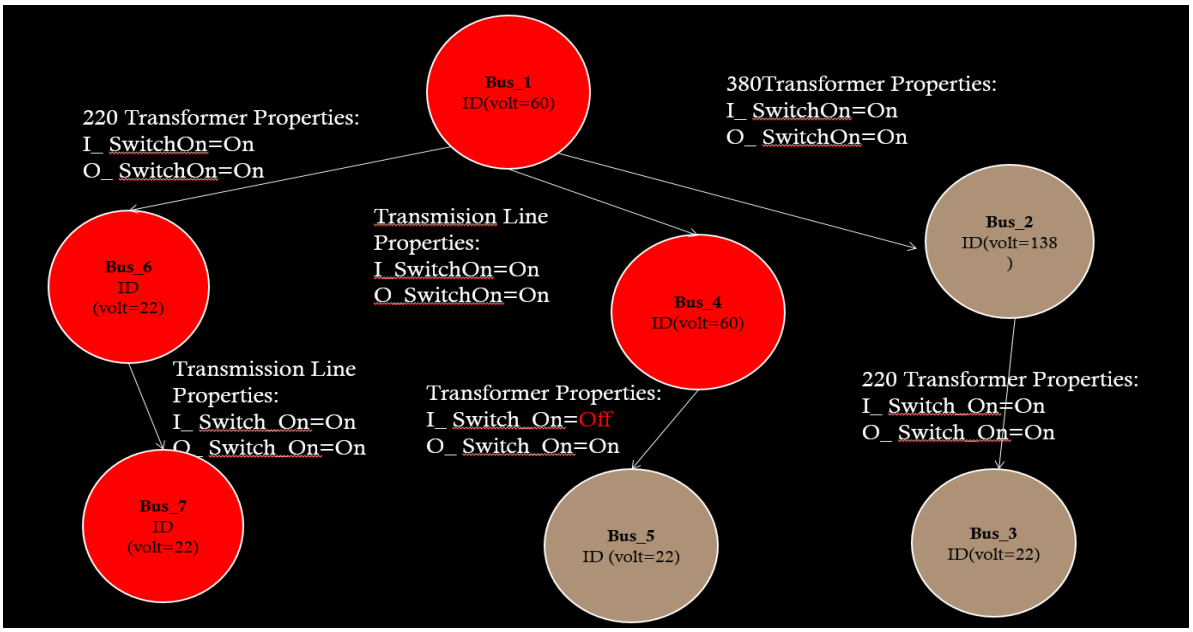


Figure 11. Initial state trigger

Elaborating on the Initial State Process in Figure. 11, Red nodes are to be seen as triggered, and brown nodes ones are not triggered (or switched OFF). Connections such as voltage drop are mapped in the Transmission lines or transformers, with "I_SwitchOn" properties and "O_SwitchOn" properties. Transformer or repeters are often times the connection between two nodes denoted with different voltage levels. Assuming that the starting node is Bus_1, the output state triggered (or switched ON) nodes should be nodes {Bus_4, Bus_6, Bus_7}. Other remaining nodes {Bus_2, Bus_3, Bus_5} are un-triggered in that the voltage of Bus_2 is lower by definition than that of Bus_1 (affecting Bus_2 and Bus_3) with connection properties indicating either "I_SwitchOn" or "O_SwitchOn" is off (affect Bus_5).

### 3.3.3 State of Triggered Event

Fig. 12 shows the architecture of initial triggered of power grid network. Before the state is triggered. A Bus_ID (node), needs to check whether the state is in the Queue Structure (Hash_Map_index_1) or not. If this bus happens to be already in queue, then there is no need to search further and it comes to a halt. This application is effective when testing on millions of nodes. The only problem at hand then is that it takes a long time. Because realworld power

grid networks have over millions of nodes, we must implement a more efficient initial triggered state approach.

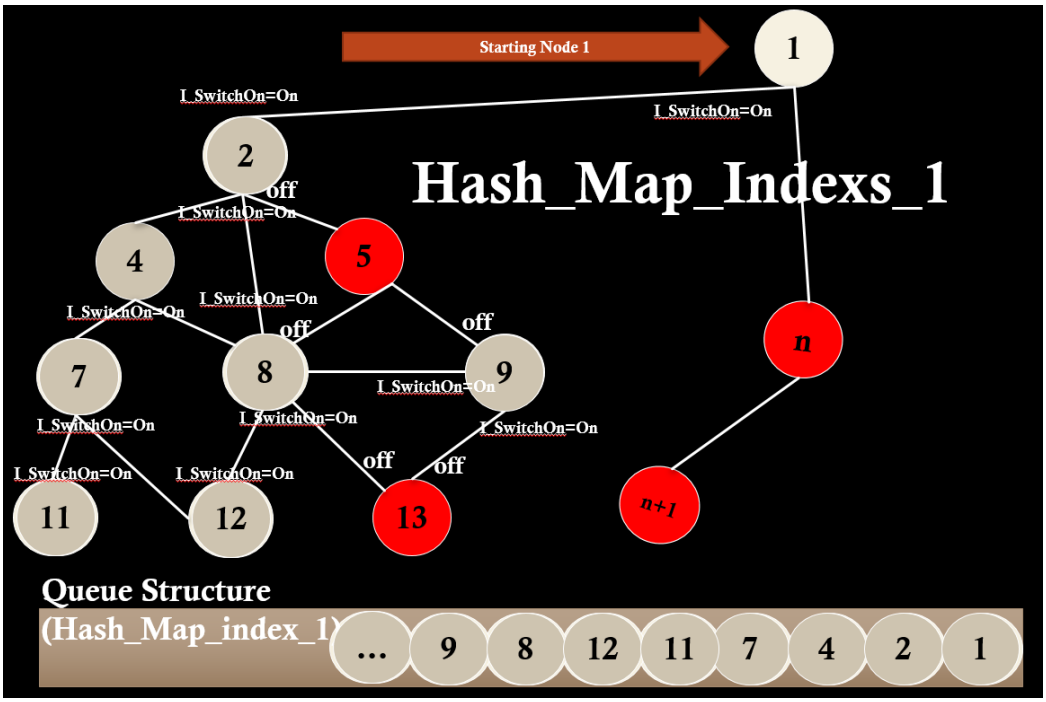


Figure 12 Node 1 initial On/Off Switches trigger of Grid network.

Initial state triggers utilizes one node as the starting node, which is the exact partition key attribute, making it unique throughout the virtual key space, then applies Breadth-First search to this node to discovers all the adjacent and proxy nodes which satisfy state conditions given by the query. The subtree for each following node that is adjacent to the starting node is also stored in a HashMap (Hash_Map_index_1). Thus, the graph is divided into an initial node and a group of HashMap indexes. In each HashMap, we use Depth-First as an equality check in order to get the properties of the ID nodes (‘I_SwitchOn’, ‘O_SwitchOn’ and voltage drop) between two buses (Nodes), then chronologically add the satisfied state triggered buses to the state Queue Structure (Hash_Map_index_1).

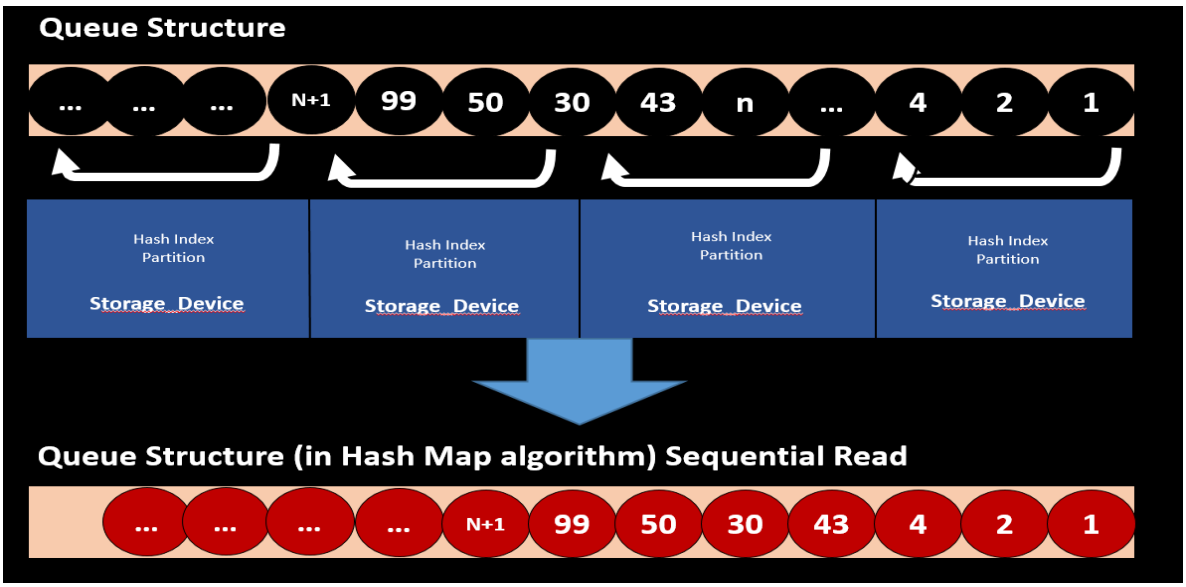


Fig. 13. Initial on multi-threading/partitioning

At scale, Traversal API provided by Neo4j implements out of the bag "BreadthFirst" function to go through all graph nodes within the partitioned key. We can alter and redesign the HashMap function which can save nodes from Traversal API. In addition, a REST APIs, See Figure 14, can very well be used to obtain all the nodes from HashMap, which demonstrates the data flow of the model in question.

Now, with all this, we can start paving the way into Cyber Security and what it means for us in a high-tech driven world.

#### 3.3.3.1 Cyber Vulnerabilities and Attacks

- *Attacks*
  - **Definition:** 'An assault on system security that derives from an intelligent threat, i.e., an intelligent act that is a deliberate attempt (especially in the sense of a method or technique) to evade security services and violate the security policy of a system.' [14]

### 3.4 Software Solutions and Components

#### 3.4.1 Information Systems topologies - Smart Grid

Networks that adhere to the TCP/IP framework are easily able to identify basics tagged payloads of an information system running all over a “Smart grid” Scenario. Information systems actually deal with processing, storing and sharing data, often times millions of millions of packets being sent back and forth. To do so in a secure manner, it requires communication technologies to be effective. Nevertheless, these technologies also require an infrastructure and protocols to be understood by all computer networks within the system, for example, Modbus from the PLC’s are configured differently than MTU’s and much more differently than Web interfaces. A client, by definition of the TCP/IP protocol is everything that consumes services. A server, is a configured computer that if root access is acquired, the Server (computer) can be used as a website for dynamic or static content or email systems can be hosted. One of the most commonly used extensions is FTP to Access File System or SMTP for email in order to make communication among URL and IP. For us to come full circle, it will need something called a domain name system or DNS. In basic terms, a client does a request for the specific website “rwth-aachen.de”, where the DNS server will solve the URL by contacting and mapping the hostname with the IP address and specifically Geo-locate the destination server.

#### 3.4.2 Server Side

A server as mentioned is a computer which sole purpose is devoted to host services that could very well be distantly accessible, such as database, email, file storage or DNS. A server can be installed locally within the network, or it can be rented from On-Demand cloud computing companies as explained in the Cloud. Apart from the technical features and functionalities related to a server, there are two main distinctive notions. First virtual servers, that means that the resources of one device is shared between several clients hosted on the same physical device. It is generally, independent of the Cloud provider, the most affordable plan. Second, one can also have dedicated server corresponding to a complete device that is solely rented to one client. Moreover, these servers can be either wired or wirelessly connected. Finally, wireless integrated network interfaces are the actual microcontrollers that equip most common connected objects between client and server, in spirit to RTUs and the SCADA Systems [13]. There are numerous devices that constitute a computer intercommunication network, such as hubs, switch and router.

### 3.4.3 Conditional Topological Search Analysis

To reiterate figure 12, The goal of system that uses a conditional topological search is to store the entire structure into graph database inside our Data Centre and perform conditional queries. The input of this model consists of a specified node (bus_number), the path number to be traversed from starting node to some termination conditions. Common termination conditions include incoming at the input node, reaching the node with status “stop” or going through a connection with either “I_Switch_On” or “O_Switch_On” off. The search will also be stopped when reaching to a particular node (bus_number) in some specific areas. In real power grid networks, some buses are managed and often times belong to a specific power station, such as central stations which are usually generating electricity.

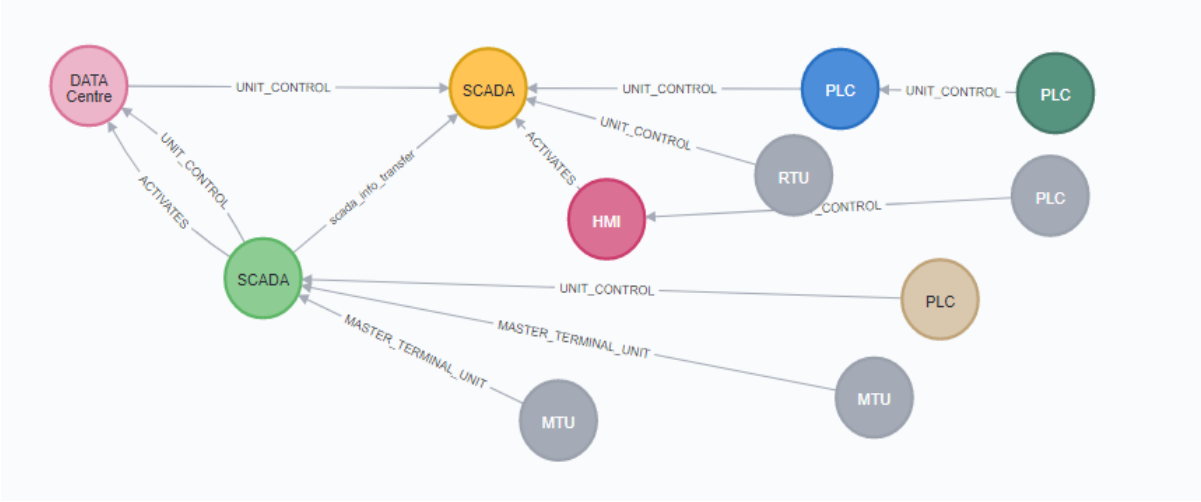


Fig. 7 Illustrates the Topological Structure of a Classical Data Centre to process information within a Subnet.

## 4. Approach, Results and Discussion

### 4.1 Understanding the Blackbox sequential steps

As data is processed and stored, the following various key concepts are explained to enhance the understanding of all the intermediate steps that are involved with regards of the transferred-managed and filtered data so that further analysis can be implemented. A mental model described as a "Blackbox of Sequential Steps".

#### 4.1.1 Link analysis explores relationships in cybersecurity

These are many applications sorts within the domain of cybersecurity; one such example is "**Link Analysis**" [12] which embodies the creation of the links between available timestamp logs to aid response-time and incident examination. It is vital from a security defense point of view, to operate and to understand links within the confines of the Network itself.

#### 4.1.2 Positives and Negatives

Scientists and staff in general can occasionally make mistakes or misinterpret the flow of data. One recurrent mistake that people do is making conclusions that something is to be determined as true when it is in fact false or conversely, concluding that roughly it is false when it is actually true. In this domain, a **false positive** is when people deduce something to be true when it is actually false (type I error). Also, a false positive can be seen as a "false alarm." Taking into account this, a **false negative** is saying something is false when it is actually true (type II error) therefore, false negative would indicate that something was not detected; something was in other words, missed. In the state-of-the-art technology, methods of fraud detection regularly fail to diminish these losses since they execute discrete analyses that are vulnerable of becoming false positives and negatives for that matter. Having this knowledge at hand, increasingly sophisticated impostors develop, often times at scale, a variety of ways to exploit and weaken the discrete analysis algorithms [12]. Graph databases, have built in functionality that offer methods of discovery fraud rings and other composite scams with a high level of precision through link analysis, and if they are implemented correctly, they are capable of ending fraud scenarios in real-time.

| Relationship between reality and hypothesis decisions | | |
|---|---|---|
| **Decision** | **Reality/Nature** | |
| | **Hypothesis True** | **Hypothesis False** |
| **Hypothesis Accepted** | True Positive (correct outcome) | False Positive |
| **Hypothesis Rejected** | False Negative | True Negative (correct outcome) |

Figure 15, Relationship between reality and hypothesis decisions

Before walking into the given results and Discussion, it is worth mentioning state-of-the-art high-end development infrastructure already in the market. It is wise for simulations to passively collect network data from IoT devices within the network environment and transform that data so that it can be mapped into a graph database in order for it to be modelled and investigated by similarity algorithms and other techniques is of paramount importance for Cybersecurity. Using 4.2 Analysis this yields the following:

In short, using Bayes' Rule

Pr($x$) function, probability of targeted event $x$

Pr(intrusion) = 0.8 (80% of intrusion happened)

Pr(x|y), probability of x given y

Conditional probability

Pr(intrution|Detection) = 0.6

60% chance of intrusion given you have a detection

Bayes' Rule (of conditional probability)

Pr(T) is attack probability, 1/10,000

Pr(T) = 0.0001

Pr(F) is probability of event flagging, unknown but useful

Pr(F|T) is 99% accuracy (higher than most techniques if implemented correctly)

Pr(F|T) = 0.99,  Pr(!F|T) = 0.01, Pr(F|!T) = 0.01, Pr(!F|!T) = 0.99

Deriving Pr(F) Pr(F) = Pr(F|T)*Pr(T) + Pr(F|!T)*Pr(!T) ‣ Pr(F) = (.99)(.0001) + (.01)(.99999) = .010098

Now, actually 99% accurate detector Pr(F) leads to …

1% possible accurate detection prediction.

With 99 false positives per true positive

$$Pr(T|F) = \frac{Pr(F|T)\ Pr(T)}{Pr(F)}$$

| IoT Device | Attack P(T) | Detector Pr(F) | Accuracy Pr(F\|T) | True Positives P(T\|F) |
|---|---|---|---|---|
| Laptop1 | 0.1 | 0.3874 | 0.6543 | 0.17178 |
| Laptop2 | 0.001 | 0.01098 | 0.9909 | 0.090 16459 |
| Laptop3 | 0.1 | 0.108 | 0.99 | 0.911 66756 |
| Laptop4 | 0.00001 | 0.00002 | 0.99999 | 0.566 |

Figure 14, Various Probabilities using Bayes' Rule

By estimating where data is coming from and how it is associated, we can work towards analysis of the network traffic that is generated by the IoT device, is the most expected progression. The scalability issue will be determined by the graph database itself to store and examine more than one network dataset at the time. We should recall upon that the primary goal is to efficiently examine the modelled data to classify and isolate malicious acts in the hosted network, such as Figure 14, the compromised IoT device with high probability was Laptop3. The analysis will be fixated on detecting the types of attacks in a simulated

environment. These attacks are typical compromises the IoT device, after being infected with malware such as Mirai [12].

### 4.2 Understanding the Dataflow within the Blackbox

- Once data is composed and processed by the IoT Hub / Network Gateway, which very well be a router or DNS server, it is to be sent to a Web Service / API for further analysis. The API, that receives the data, places that processed new data onto a queue for static storage utilizing the caching features of the database and then eventually transformed into the desired graph schema ready for visualization and further analysis. Constantly running the similarity algorithms to identify malicious activity could be one approach, if the API is running so called Cronjobs (Automatic Asynchronous Chronological Calls). [12]
- The API obtains the new (already processed) data and places it onto another queue, typically using python open source software called celery utilizing a Redis in-memory storage system as a message broker manager, which is one of its most extensively used "filters" for processing live streaming.

Secondly, to classify the threats using "link analysis" the Neo4j native browser user interface lends itself for visualization. The Neo4j native browser user interface permits direct access to the database itself and the queries can be executed and analyzed in order to find threats based known metrics. Additionally, the Neo4j graph database has built in support for the similarity algorithms which in context aim to aid in the analysis of Anomalies Detection and Cybersecurity.

### 4.3 Similarity Algorithms

After the data has been stored in Neo4j as described in **4.2 Understanding the Dataflow within the Blackbox**, the analyzer program, will be used and altered to compute the similarity of the dataflow, by looking at a given time interval of network traffic and associating it to prior network traffic for that IoT device, using as for example Wireshark, which is the world's

leading and widely-used network protocol “analyzer” to scan data traffic throughout any system, including routers and servers. The analyzer will utilize some of the built-in algorithms found in Neo4j’s add-on packages, more precisely the similarity algorithms known as cosine, pearson, and jaccard. These algorithms make available possible detections and helps to identify irregularities in a dataflow.

In order to further comprehend the studied algorithms, we must all come to terms, and be aware of the language used and adhere to their lexicon. The [12]is a good compact basis for some of these strategies in terms of standards for the World Wide Web.

- Generation is the event of when an entity is created for the first time.
- Usage is the beginning of utilizing an entity for an activity.
- Communication is described as the exchange of a specific entity by two activities.
- An agent is something that bears some form of responsibility for an activity taking place, for the existence of an entity, or for another agent's activity.
- Finally, attribution which illustrates an association of an entity to an agent. [12]

The mental model is to have a strong approach, with the help of the similarity algorithms of cosine, pearson and jaccard, whose sole purpose is to identify potential TCP Flood Attacks, UDP Flood Attack, and even ICMP flood attack all the way to port scanning. [12]

### 4.3.1 Interpretation of the Dataflow

There are three different algorithms that will be used as mentioned to analyze the relationships of the network are cosine, pearson, and jaccard, which can also be used for “nearest neighbors”. Every single one of these algorithms analyzes the dataflow to some extent in a diverse way and their calculated indexes and coefficients have by enlarge different numerical meanings, which will be furthered explained.

- **Cosine Similarity Equation**

$$S = \cos(\theta) = \frac{A \cdot B}{\|A\|\|B\|} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2}\sqrt{\sum_{i=1}^{n} B_i^2}}.$$

  - Cosine Similarity equation where A and B, given by the above representation of the equation, are two sets. – [11]. Cosine similarity algorithm will generate a value that is within the range between 0 and 1.

| "from" | "to" | "similarity" |
|---|---|---|
| "Moser" | "Vince" | 0.9542262139256075 |

```
CREATE (omer)-[:OPERATES {score: 9}]->(firewall)
CREATE (omer)-[:OPERATES{score: 7}]->(os_version)
CREATE (omer)-[:OPERATES{score: 8}]->(rtu1)
CREATE (omer)-[:OPERATES{score: 1}]->(mtu1)

CREATE (vince)-[:OPERATES {score: 10}]->(subnet1)
CREATE (vince)-[:OPERATES{score: 6}]->(firewall)
CREATE (vince)-[:OPERATES{score: 2}]->(rtu1)

CREATE (moser)-[:OPERATES {score: 8}]->(subnet1)
CREATE (moser)-[:OPERATES {score: 7}]->(database)
CREATE (moser)-[:OPERATES {score: 9}]->(firewall)
CREATE (moser)-[:OPERATES{score: 3}]->(os_version)

CREATE (ulbig)-[:OPERATES {score: 10}]->(subnet2)
CREATE (ulbig)-[:OPERATES{score: 10}]->(database)
CREATE (ulbig)-[:OPERATES {score: 7}]->(os_version)
CREATE (ulbig)-[:OPERATES {score: 9}]->(mtu1)

CREATE (taufenbach)-[:OPERATES{score: 9}]->(subnet2)
CREATE (taufenbach)-[:OPERATES {score: 7}]->(os_version)
```

  - This denotes that 0 represents that the sets are “dissimilar” and 1 is “similar”. If the given value is within the range closer to zero, it will be identified as an “anomaly”, because the IoT device's usual communication over a times spam should become highly predictable, which should be a range of values closer to one. Once of course, the traffic of the IoT device is compared to prior dataflow of the same IoT device. The higher the score the higher the times it was accessed or logged in. the Score for example can be mapped from an external API such as a NAT-Gateway, which can identify every time the was sent a packet from Outside the Network, with an invalid IP address.

- **Pearson Similarity Equation**

$$similarity(A,B) = \frac{\mathrm{cov}(A,B)}{\sigma_A \sigma_B} = \frac{\sum_{i=1}^{n}(A_i - \overline{A})(B_i - \overline{B})}{\sqrt{\sum_{i=1}^{n}(A_i - \overline{A})^2 (B_i - \overline{B})^2}}$$

  - Pearson Similarity equation where A and B, given by the above representation of the equation, are two sets. [11]. Pearson similarity algorithm generates a value that ranges between -1 and 1, where coefficient that is derived which is greater than zero and closer to 1 would mean that the two vectors are positively correlated or annexed, and a 0 means that the two vectors are independent or have no correlation at hand, and as it gets closer to -1 mean negative correlation or that the two vectors are "dissimilar".

```
MERGE (vince:Software {name:'Vince'})
MERGE (moser:Software{name:'Moser'})
MERGE (ulbig:Software {name:'Ulbig'})
MERGE (omer:Software {name:'Omer'})
MERGE (taufenbach:Software{name:'Taufenbach'})
MERGE (hyemin:Software{name:'Hyemin'})

MERGE (laptop1:Person_used {name: 'Laptop1'})
MERGE (laptop2:Person_used {name: 'Laptop2'})
MERGE (laptop3:Person_used {name: 'Laptop3'})
MERGE (laptop4:Person_used {name: 'Laptop4'})
MERGE (laptop5:Person_used {name: 'Laptop5'})

MERGE (laptop1)-[:RATED_BY_PRIORITY {score: 2}]->(omer)
MERGE (laptop1)-[:RATED_BY_PRIORITY {score: 2}]->(ulbig)
MERGE (laptop1)-[:RATED_BY_PRIORITY {score: 3}]->(moser)
MERGE (laptop1)-[:RATED_BY_PRIORITY {score: 6}]->(taufenbach)

MERGE (laptop2)-[:RATED_BY_PRIORITY {score: 6}]->(omer)
MERGE (laptop2)-[:RATED_BY_PRIORITY {score: 7}]->(ulbig)
MERGE (laptop2)-[:RATED_BY_PRIORITY {score: 8}]->(moser)
MERGE (laptop2)-[:RRATED_BY_PRIORITY {score: 9}]->(vince)

MERGE (laptop3)-[:RATED_BY_PRIORITY {score: 7}]->(omer)
MERGE (laptop3)-[:RATED_BY_PRIORITY {score: 9}]->(ulbig)
MERGE (laptop3)-[:RATED_BY_PRIORITY {score: 3}]->(vince)
MERGE (laptop3)-[:RATED_BY_PRIORITY {score: 4}]->(moser)

MERGE (laptop4)-[:RATED_BY_PRIORITY {score: 8}]->(moser)
MERGE (laptop4)-[:RATED_BY_PRIORITY {score: 1}]->(vince)
MERGE (laptop4)-[:RATED_BY_PRIORITY {score: 10}]->(taufenbach)
MERGE (laptop4)-[:RATED_BY_PRIORITY {score: 10}]->(hyemin)

MERGE (laptop5)-[:RATED {score: 9}]->(vince)
MERGE (laptop5)-[:RATED {score: 7}]->(matrix)
MERGE (laptop5)-[:RATED {score: 7}]->(omer)
MERGE (laptop5)-[:RATED {score: 9}]->(hyemin)
```

```
MATCH (p1:Person_used {name: 'Laptop2'})-[intrution:RATED_BY_PRIORITY]→(software)
WITH p1, gds.alpha.similarity.asVector(software, intrution.score) AS p1Intrution
MATCH (p2:Person_used)-[intrution:RATED_BY_PRIORITY]→(software) WHERE p2 ◇ p1
WITH p1, p2, p1Intrution, gds.alpha.similarity.asVector(software, intrution.score) AS p2Intrution
RETURN p1.name AS from,
p2.name AS to,
gds.alpha.similarity.pearson(p1Intrution, p2Intrution, {vectorType: "maps"}) AS similarity
ORDER BY similarity DESC
```

| "from" | "to" | "similarity" |
|---|---|---|
| "Laptop2" | "Laptop4" | 1.0 |
| "Laptop2" | "Laptop1" | 0.39605901719066977 |
| "Laptop2" | "Laptop3" | -0.4851346705148632 |

  - It is predictable that values that ranges equal to or less than zero would be a revealing sign that a given IoT device has been compromised, everything else would be considered of regular traffic. Similar in spirit as the Cosine Equation previously discussed, the higher the score, the higher the times the data was accessed on file. the Score for example can be mapped from an external

API such as a NAT-Gateway, which can identify every time the was sent a packet from Outside the Network, with an invalid IP address.

- **Jaccard Similarity Equation**

$$J(A,B) = \frac{|A \cap B|}{|A \cup B|} = \frac{|A \cap B|}{|A| + |B| - |A \cap B|}$$

  - Jaccard Similarity equation where A and B, given by the above representation of the equation, are two sets. [11] Now, Jaccard Similarity algorithm generates an index or "similarity" coefficient by comparing two sets of dataflow and deciding if is an overlap between the two sets. The coefficients do range between 0 and 1.

```
CREATE
(subnet1:Intrution {name:'Subnet1'}),
(database:Intrution {name:'Database'}),
(firewall:Intrution {name:'Firewall'}),
(subnet2:Intrution {name:'Subnet2'}),
(os_version:Intrution {name:'Os_version'}),

(vince:detective {name: 'Vince'}),
(omer:detective {name: 'Omer'}),
(moser:detective {name: 'Moser'}),
(ulbig:detective{name: 'Ulbig'}),
(taufenbach:detective {name: 'Taufenbach'}),

(omer)-[:HAVE_ACCESS_TO]->(firewall),
(omer)-[:HAVE_ACCESS_TO]->(os_version),

(vince)-[:HAVE_ACCESS_TO]->(subnet1),
(vince)-[:HAVE_ACCESS_TO]->(firewall),

(moser)-[:HAVE_ACCESS_TO]->(subnet1),
(moser)-[:HAVE_ACCESS_TO]->(database),
(moser)-[:HAVE_ACCESS_TO]->(firewall),

(ulbig)-[:HAVE_ACCESS_TO]->(subnet2),
(ulbig)-[:HAVE_ACCESS_TO]->(database),
(ulbig)-[:HAVE_ACCESS_TO]->(os_version),

(taufenbach)-[:HAVE_ACCESS_TO]->(subnet2),
(taufenbach)-[:HAVE_ACCESS_TO]->(database)
```

```
MATCH (p1:Detective {name: 'Ulbig'})-[:HAVE_ACCESS_TO]→(random_access)
WITH p1, collect(id(random_access)) AS p1Download
MATCH (p2:Detective)-[:HAVE_ACCESS_TO]→(limited_access) WHERE p1<> p2
WITH p1, p1Download, p2, collect(id(limited_access)) AS p2Download
RETURN p1.name AS from,
       p2.name AS to,
       gds.alpha.similarity.jaccard(p1Download, p2Download) AS similarity
ORDER BY to, similarity DESC
```

  - Having this, the closer it is to 1 means that the more similar the two sets of dataflow are. So, indexes computed closer to 0 for a given time gap would be anticipated to be compromised or hacked.

| "from" | "to" | "similarity" |
|---|---|---|
| "Ulbig" | "Moser" | 0.2 |
| "Ulbig" | "Omer" | 0.25 |
| "Ulbig" | "Taufenbach" | 0.6666666666666666 |
| "Ulbig" | "Vince" | 0.0 |

### 4.4 Concluding results

Generally, the use of graph models and graph databases is a trend that is exponentially growing. The argument initially did highlight how the Grid system became a network and with graph databases we can model clear pathways to effective Cybersecurity database visualization. From a practical point of view, if used to its full extent, most graph database platforms do provide a rich ecosystem for power grids, and one may agree is easy to scale, having the ability to show relationships between several nodes that can also be clearly monitored. Furthermore, when the intention is to implement a power grid system, it is crucial that the database itself considers the specific requirements given by the power grid itself. Generic graph databases are limited in their ability to scale and more so provide a viable answer for a large-scale power grids, however if used with the right data points and fused with different layers as explained in **4.2** and **2.1.2.2**, graph databases in general have clear lead over inflexible table-based SQL data models. The scores can very well be placed as Wild_Card or as Super_Globals using any procedural programming language, such as php, python or javascript for that matter, that could alter the count asynchronously at any given moment, in response to certain events, such as pinning, log files, emails-being-sent and more.

# 5 Conclusion

## 5.1 Summary

As we consider and further enhance the Smart Grid Architecture that allocates efficiently and well monitored energy, it is optimal to have constant visibility of the data transfer and overall energy consumption. If we look back in history, Smart Grids worked with operators and staff that were behind levers and dials of transformers or generator in order to monitor their current state and adjusted it if needed. Some of our engineers and Staff were eventually replaced by sensors to monitor gas consumption, air and electricity production and forecast possible problems in the Cybersecurity domain that might arise in order to anticipate and prepare for such an event. The arrival of Cloud Computing and Cybersecurity to some extent changed the manner by in which we transferring data from our local infrastructure such as servers, RTU's or even PLC's to distant servers and calculation facilities operated by third-party managed companies. This new ensemble of collective parts of the Grid permits distant monitoring and control making it a very powerful aspect to push even further any need of Smart Grids and for Cybersecurity.

With regards of the composition of the Graph Model, systems such as Smart Grids are constituted in such a way that presents data to an administrator in various forms and formats in order to control the ongoing process that connects PLC or RTU to supervisory computers and its Security [13]. There is a current trend to use devices that have already an integrated controller and a network interfaces which make it easier to the consumer and to the administrator for cybersecurity. Walking into the domain of performance engineering and Human Factors Engineering which by definition considers our human capabilities and actual limitations. The Main take away is to remember about Smart Grid is it structures its controller with sensors and actuators within the network infrastructure, forwarding packets or frames of data, using supervisory computers, that will eventually be used for processing and producing visualization all in a Real-Time scenario. Now, walking into the real world where money is involved and a budget to consider, being able to structure and understand a Smart Grids system is a plus, though it remains quite difficult to talk about smart grids without mentioning its self-automated and self-intrusion detection aspects which could be realized as it is explored beyond this thesis.

## 5.2 Future work

As we have an overview of the overall topology, the International Telecommunication Union also acknowledges the functional capabilities of Smart Grids and how to deal with some of its imbedded fault management problems, as we overcome the challenges such as configuration and security features that could face authorization, authentication, device integrity validation in its intrusion detection attacks. Smart Grid are the technology of tomorrow, being designed today, which has been already evolving the last 40 years, with the nuance of wireless communication protocols and miniaturization of hardware controllers. It became a deployed network of sensors and actuators that can funnel data to a device so that administrators are able to have fine grained access control of the overall topology. To conclude, this further exciting extension portion of this last chapter, is to highlight the hunger for automated Smart Grids systems. My work future steps would include:

1. Impact of dataset size on State Triggered performance
2. Impact of multiple threads on State Triggered performance
3. performance using single thread on Neo4j and Other Databases.
4. performance using multiple threads (multiple cores) on Neo4j and other databases.
5. Consequently, multithread communication on Core API in Neo4j could be more efficient than Other Databases.

### 5.2.1 Extension of the possible future work

1. As data is batched and processed, parsed in an JSON or YAML file from the graph database, it can be put into an Elasticsearch database. ***Elasticsearch*** is a type of NoSQL database positioned on time-based storage, storing and maintaining all the computed algorithms so that similar data stands out. So that, as the data becomes critical, there might be a need for “Kibana” in order to to visualize the outcomes of the data kept in the database.
2. ***Kali Linux*** as the operating system. Kali Linux is not only considered for penetration testing as one of the many Linux distributions, but it is also used for ethical hacking and network security management. Kali has vast documentation regarding execute commands concerning topics such as information vulnerability analysis, gathering stress testing, which can also be used for DDoS attacks and nmap vulnerability scans.

3. ***Anomaly Detection based on Machine Learning.*** Besides graph analysis, arithmetical and machine learning methods are being put in place as it is considered the identification of attack vectors. Data is ultimately processed by several Support Vector Machines (SVMs) and based on the normal user behavior, being put into sequential models such as Convolutional Neural Networks (CNNs) and Deep neural networks (DNNs) for progressively find patterns of anomalies at scale.

**Bibliography**

[1] Report citation IRENA (2018), Global Energy Transformation: A roadmap to 2050, International Renewable Energy Agency, Abu Dhabi. www.irena.org/publications.

[2] The Global Competitiveness Report 2010-2011 © 2010 World Economic Forum, Professor Xavier Sala-i-Martin Columbia University Chief Advisor of the Centre for Global Competitiveness and Performance

[3] Sino-German Energy Transition Project commissioned by Federal Ministry for Economic Affairs and Energy (BMWi) Authors: Anders Hove, Qian Wenyun, Zhao Kaiming, Nicole Kim Fuerst Deutsche Gesellschaft für Internationale Zusammenarbeit (GIZ) GmbH

[4] Subcommittee on Climate-Related Market Risk of the Market Risk Advisory Committee (MRAC) David Gillers, Chief of Staff, Office of Commissioner Behnam Bob Litterman, Chairman Leonardo Martinez-Diaz, Editor Jesse M. Keenan, Editor Stephen Moch, Associate Editor.

[5] Don Von Dollen et al. "Report to NIST on the smart grid interoperability standards roadmap". In: *Electric Power Research Institute (EPRI) and National Institute of Standards and Technology* (June 2009). retrieved from https://www.nist.gov/system/files/documents/smartgrid/ InterimSmartGridRoadmapNISTRestructure.pdf on 26 September 2020, p. 21.

[6]Janaka Ekanayake et al. *Smart Grid: Technology and Applications*. 1st. Wiley Publishing, 2012. ISBN: 0470974095.

[7] ReaserchGate. Discover scientific knowledge and stay connected to the world of science. An-overview-of-the-cloud-integrated-smart-grid-architecture-with-access-to-unlimited.png (850×563) (researchgate.net)

[8]EV Charging Behavior Analysis Using Hybrid Intelligence for 5G Smart Grid, January 2020 Electronics 9(1):80 Yi Shen, Wei Fang, Feng Ye and Michel Kadoch

[9]Sotiriadis, S., Bessis, N. and Buyya, R., 2017. Self managed virtual machine scheduling in Cloud systems. Information Sciences

[10] Osanaiye, O., Chen, S., Yan, Z., Lu, R., Choo, K.K.R. and Dlodlo, M., 2017. From cloud to fog computing: A review and a conceptual live VM migration framework. IEEE Access, 5, pp.8284- 8300

[11] Developers, Neo4J. "Neo4J." Graph NoSQL Database [online] (2012).

[12] A Direct Mapping of Relational Data to RDF, M. Arenas, A. Bertails, E. Prud'hommeaux, J. Sequeda, Editors, W3C Recommendation, September 27, 2012,

http://www.w3.org/TR/2012/REC-rdb-direct-mapping-20120927/. Latest version available at http://www.w3.org/TR/rdb-direct-mapping/.
[13] M. S. Thomas and J. D. McDonald, Power system SCADA and smart grids. Boca Raton, Florida: CRC Press, 2015. [Online]. Available: http://gbv.eblib.com/patron/FullRecord.aspx?p=1829472
[14]CISCO SYSTEMS **Cisco IoT system** 2015 - Cisco Sytems – California **In-text:** (Cisco Systems, 2015) Cisco Systems, 2015. *Cisco IoT system*. California: Cisco Sytems, pp.1-52.

**Figuren List**

Figure 2.1.1: European Electric Grid scheme
Figure 2.1.2: Simplified Smart Grid scheme
Figure 2.1.2: Fog Layer and the Cloud Layer of the Smart Grid scheme
Figure 3.1: table NoSQL
Figure 3.2: Table SQL/NoSQL
Figure 3.3 Unordered hash index Partition
Fig. 7. Illustrates the Topological Structure of a Classical Data Centre to process information within a Subnet.
Figure. 10. Topology of Graphs Relationships
Figure 11. Initial state trigger
Figure 12. Node 1 initial On/Off Switches trigger of Grid network
Fig. 13. Initial on multi-threading/partitioning
Figure 14. REST APIs provide GET and POST function for conditional search. These functions can be called either from CLI PowerShell or directly from Neo4j client.
Figure 15. Relationship between reality and hypothesis decisions